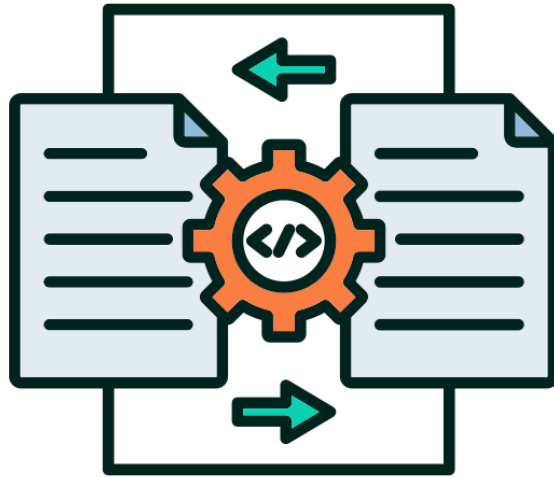




72.39 - Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial - Entrega N°3

Diseño e implementación de un lenguaje

Profesores:

Arias Roig, Ana María

Golmar, Mario Agustín

Integrantes del grupo: “JSON2SQL”

64396	Menshikoff, Katia	kmenshikoff@itba.edu.ar
63622	Santamarina Balbin, Manuel José	msantamarinabalbin@itba.edu.ar
63162	Mesa Rubio, Santiago	smesarubio@itba.edu.ar
61067	Sapino, Matías	msapino@itba.edu.ar

Índice

1. Introducción.....	2
2. Ejecución.....	2
3. Modelo Computacional.....	2
2.1 Dominio.....	2
2.2 Lenguaje.....	2
4. Implementación.....	3
3.1 Frontend.....	3
3.2 Backend.....	3
3.3 Dificultades Encontradas.....	4
5. Futuras Extensiones.....	4
6. Conclusiones.....	5
7. Referencias.....	5
8. Bibliografía.....	5

1. Introducción

El proyecto JSON2SQL tiene como objetivo principal diseñar e implementar un lenguaje que transforme estructuras JSON en consultas SQL. Este trabajo se enmarca en la asignatura Autómatas, Teoría de Lenguajes y Compiladores (72.39) y tiene como propósito aplicar conceptos teóricos de lenguajes formales y técnicas de compilación en un problema práctico, desarrollando tanto la sintaxis como la semántica de un lenguaje intermedio para la manipulación de bases de datos relacionales.

La entrega N°3 se centró en el desarrollo del backend, donde se llevó a cabo la implementación del núcleo funcional del sistema. Este componente es responsable de recibir entradas en formato JSON, validarlas mediante análisis léxico y sintáctico, y generar consultas SQL válidas. Este informe detalla el modelo computacional, la implementación técnica, los desafíos enfrentados, y las perspectivas de evolución del proyecto.

2. Ejecución

Para compilar y ejecutar el proyecto se debe hacer:

```
./run.sh
```

3. Modelo Computacional

2.1 Dominio

El dominio del sistema se encuentra en la intersección entre los formatos de datos estructurados JSON y los sistemas de bases de datos relacionales (SQL). JSON2SQL facilita la interoperabilidad entre estos dos mundos al permitir que aplicaciones modernas que utilizan JSON puedan integrarse de manera sencilla con sistemas relacionales tradicionales.

El sistema está diseñado para soportar las operaciones fundamentales de SQL:

- Inserciones (INSERT INTO) para agregar nuevos registros.
- Consultas (SELECT) para recuperar información.
- Actualizaciones (UPDATE) para modificar registros existentes.
- Eliminaciones (DELETE) para remover registros.

Además, el sistema puede ser extendido para admitir operaciones avanzadas como JOIN, subconsultas y transacciones.

2.2 Lenguaje

El lenguaje JSON2SQL acepta entradas en formato JSON con una estructura definida, que especifica la operación, la tabla objetivo y los valores o condiciones asociados. Este lenguaje se diseñó teniendo en cuenta la simplicidad y la capacidad de extensión, permitiendo que usuarios no expertos puedan escribir instrucciones claras y comprensibles.

Ejemplo de entrada y salida:

Entrada JSON:

```
{
  "SELECT": ["name", "salary"],
  "FROM": "employees",
  "WHERE": {
    "department": "sales",
    "salary": {"gt": 1000}
  },
  "GROUP BY": ["department"],
  "ORDER BY": ["name"],
  "HAVING": [
    {"COUNT": "name", "gt": 1 }
  ]
}
```

Salida SQL:

```
SELECT salary, name FROM employees
WHERE (department = sales AND salary > 1000)
GROUP BY department
HAVING COUNT(name) > 1
ORDER BY name
```

La gramática de JSON2SQL se define mediante una gramática libre de contexto (CFG), que abarca las construcciones válidas para las operaciones soportadas. La validación incluye no solo la estructura sintáctica, sino también restricciones semánticas como:

- Verificar que las columnas solicitadas existan en la tabla.
- Garantizar que los valores tengan un tipo adecuado según el esquema.

4. Implementación

3.1 Frontend

En esta etapa, no se realizaron avances específicos en el frontend. La gramática definida en la segunda entrega validaba prácticamente todos los casos de aceptación.

3.2 Backend

La implementación del backend se centró en transformar los datos proporcionados por el archivo JSON en consultas SQL válidas. Este proceso incluyó múltiples etapas cuidadosamente diseñadas para garantizar la precisión y la flexibilidad en la generación del código.

Generación de consultas SQL

El proceso de generación de consultas SQL se basa en un recorrido detallado del árbol de sintaxis abstracta (AST). A partir del tipo de operación especificada en el JSON (como CREATE, SELECT, INSERT, etc.), el sistema invoca funciones específicas que descomponen el árbol en elementos clave de una consulta SQL. Estas funciones navegan de manera recursiva por estructuras del AST, como listas de columnas, condiciones WHERE y valores para inserciones. Cada fragmento del AST se traduce en su equivalente en SQL,

ensamblando una consulta completa y correctamente formateada. Este enfoque modular y recursivo garantiza la extensibilidad del sistema para admitir nuevas estructuras y operaciones en el futuro.

Soporte para consultas anidadas

La generación de consultas anidadas representó un desafío significativo debido a la complejidad inherente de estas estructuras. Se implementaron estrategias específicas para procesar subconsultas, asegurando que fueran correctamente integradas en las consultas principales. Este desarrollo requirió ajustes en las funciones de generación de SQL para manejar correctamente la jerarquía y los contextos de las subconsultas.

3.3 Dificultades Encontradas

Adaptación entre Frontend y Backend

Uno de los desafíos principales fue alinear las salidas del frontend con los requisitos del backend. Algunas estructuras generadas por el frontend requerían ajustes para que el backend pudiera procesarlas de manera efectiva. Este proceso involucró una estrecha colaboración entre ambas etapas para garantizar una integración fluida y coherente.

Manejo de Consultas Anidadas

La implementación de consultas anidadas presentó dificultades debido a la complejidad de procesar múltiples niveles de anidación. Este problema se abordó extendiendo las capacidades del generador de SQL para manejar de manera recursiva subconsultas y su contexto. Aunque este enfoque agregó complejidad al desarrollo, también mejoró la robustez del sistema.

Escalabilidad del Generador de Código

La necesidad de soportar múltiples tipos de consultas y estructuras del JSON aumentó significativamente la complejidad del generador de código. Este desafío se resolvió mediante un diseño modular que permite añadir nuevas funcionalidades sin afectar las existentes. Sin embargo, asegurar esta modularidad requirió una planificación cuidadosa y revisiones iterativas.

5. Futuras Extensiones

El proyecto tiene un alto potencial de expansión. Entre las funcionalidades planeadas para el futuro se incluyen:

- Soporte para Transacciones: Incorporar comandos como BEGIN TRANSACTION, COMMIT y ROLLBACK.
- Operaciones Avanzadas: Extender el soporte a operaciones complejas como combinaciones de tablas (JOIN) y subconsultas.

- Compatibilidad con Dialectos SQL: Adaptar el generador para soportar sintaxis específicas de sistemas como PostgreSQL, MySQL o SQL Server. Todo partiendo de un mismo JSON.

6. Conclusiones

El desarrollo del backend de JSON2SQL representa un avance significativo en el objetivo del proyecto, demostrando que es posible convertir estructuras JSON a comandos SQL válidos mediante un flujo robusto de análisis léxico, sintáctico y semántico. Este sistema es funcional, extensible y adaptable a escenarios reales, proporcionando una herramienta que puede ser utilizada en aplicaciones modernas que requieran interactuar con bases de datos relacionales.

Aunque se enfrentaron varios desafíos, el equipo logró implementar una solución funcional, sentando las bases para futuras extensiones y mejoras del sistema.

7. Referencias

- ANTLR Documentation. (<https://www.antlr.org>)
- JSON Schema Specification. (<https://json-schema.org>)
- Stack Overflow, Discusiones sobre ANTLR y SQL Parsing.

8. Bibliografía

- Aho, A. V., Sethi, R., & Ullman, J. D. Compilers: Principles, Techniques, and Tools. Pearson.
- Melton, J., & Simon, A. R. SQL:1999 - Understanding Relational Language Components. Morgan Kaufmann.
- Wirth, N. Algorithms + Data Structures = Programs. Prentice Hall.